Tab 1

Machine Learning Algorithms

Supervised Learning

Supervised learning involves training a model on a labeled dataset, where the algorithm learns to map inputs to a known output.

- **Linear Regression:** A simple algorithm that models a linear relationship between inputs and a continuous numerical output variable. It is highly interpretable and fast to train, though it assumes linearity and is sensitive to outliers.
- **Logistic Regression:** Used for classification, this algorithm models the relationship between inputs and a categorical output (typically 1 or 0). It is less prone to overfitting when regularized but also assumes linear relationships.
- **K Nearest Neighbors (KNN):** An algorithm that classifies data points based on the proximity to neighboring samples. In audio processing, this can be used for cross-similarity tasks to find samples with similar characteristics.
- **Support Vector Machine (SVM):** A powerful classifier that finds the optimal hyperplane to separate classes in high-dimensional space.
- **Naive Bayes Classifier:** A probabilistic classifier based on Bayes' Theorem, often used for text and simple categorical classification.
- **Decision Trees:** These models use a series of "if-then" rules based on features to produce predictions. While easy to interpret, they are prone to overfitting.

Ensemble Algorithms

Ensemble methods combine multiple models to improve overall predictive performance and accuracy.

- **Bagging & Random Forests:** Random Forests combine the output of multiple decision trees to reduce overfitting and increase accuracy.
- **Boosting & Strong Learners:** Algorithms like Gradient Boosting, XGBoost, and LightGBM build an ensemble of weak learners (typically trees) to capture non-linear relationships and provide highly accurate results.

Neural Networks & Deep Learning

Advanced models designed to recognize patterns. For audio applications, Convolutional Neural Networks (CNNs) are particularly effective for tasks like audio classification and feature extraction.

Unsupervised Learning

Unsupervised learning finds hidden structures or patterns in input data without pre-existing labels.

- **Clustering / K-means:** Groups data points into clusters based on similarity. This is useful for temporal segmentation and sub-segmenting audio files by feature similarity.
- **Dimensionality Reduction:** Techniques used to reduce the number of random variables under consideration.
    - **Principal Component Analysis (PCA):** A method for simplifying complex data by reducing its dimensions while preserving as much variance as possible.

Tab 2

Machine Learning Algorithms: Python Code Examples

The following examples use common Python libraries like `NumPy` for data manipulation and `scikit-learn` (aliased as `sklearn`) and `TensorFlow`/`Keras` for implementing the algorithms mentioned in your document.Supervised LearningLinear Regression[1]

This example demonstrates how to train a model to predict a continuous output.

Python

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# 1. Prepare synthetic data
X = np.array([[1], [2], [3], [4], [5], [6]])  # Input feature
y = np.array([2, 4, 5, 4, 5, 6])              # Continuous output
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 2. Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# 3. Make and evaluate predictions
predictions = model.predict(X_test)
print(f"Predictions: {predictions}")
print(f"Model Coefficient (Slope): {model.coef_[0]}")
```

Logistic Regression (for Classification)[1]

This example demonstrates how to use the algorithm for a binary classification task.

Python

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Prepare synthetic data (e.g., classifying a tumor as malignant (1) or
benign (0))
X = np.array([[10, 5], [1, 2], [15, 8], [2, 1], [3, 4]])  # features
(size, texture)
y = np.array([1, 0, 1, 0, 0])                             # categorical
output (1 or 0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
random_state=42)

# 2. Initialize and train the model
model = LogisticRegression()
```

```
model.fit(X_train, y_train)

# 3. Make and evaluate predictions
predictions = model.predict(X_test)
print(f"Test Accuracy: {accuracy_score(y_test, predictions)}")
```

K Nearest Neighbors (KNN)[1]

This example classifies new data points based on the features of their neighbors.

Python

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load a standard dataset
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 2. Initialize and train the model (using 3 neighbors)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# 3. Make and evaluate predictions
predictions = knn.predict(X_test)
print(f"KNN Accuracy: {accuracy_score(y_test, predictions)}")
```

Ensemble AlgorithmsBagging & Random Forests[1]

Random Forest builds multiple decision trees to improve accuracy and robustness.

Python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load a standard dataset
wine = load_wine()
X, y = wine.data, wine.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# 2. Initialize and train the model (using 100 decision trees)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 3. Make and evaluate predictions
```

```
predictions = model.predict(X_test)
print(f"Random Forest Accuracy: {accuracy_score(y_test, predictions)}")
```

Neural Networks & Deep LearningSimple Feedforward Neural Network (using Keras/TensorFlow)[1]

This demonstrates a basic deep learning model for classification. Your context mentions that a course teaches deep learning with neural networks and TensorFlow.[2][3][4][5]

Python

```
import tensorflow as tf
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

# 1. Prepare synthetic data for a non-linear problem
X, y = make_circles(n_samples=1000, noise=0.1, factor=0.5,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 2. Build the sequential model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation='relu', input_shape=(2,)), #
Input layer with 2 features
    tf.keras.layers.Dense(5, activation='relu'),                     #
Hidden layer
    tf.keras.layers.Dense(1, activation='sigmoid')                  #
Output layer for binary classification
])

# 3. Compile and train the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, verbose=0)

# 4. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Neural Network Accuracy: {accuracy:.4f}")
```

Unsupervised LearningClustering / K-means[1]

This example groups similar data points into a specified number of clusters (`k`).

Python

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. Prepare synthetic data with 3 distinct groups
```

```
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.60,
random_state=42)

# 2. Initialize and train the model (looking for 3 clusters)
kmeans = KMeans(n_clusters=3, random_state=42, n_init='auto')
kmeans.fit(X)

# 3. View the assigned cluster labels for each data point
cluster_labels = kmeans.labels_
print(f"First 10 cluster assignments: {cluster_labels[:10]}")
print(f"Coordinates of the 3 final cluster
centers:\n{kmeans.cluster_centers_}")
```

Principal Component Analysis (PCA)[1]

This example reduces the dimensionality of a dataset while retaining most of the variance.

Python

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
import pandas as pd

# 1. Load a standard dataset
iris = load_iris()
X = iris.data

# 2. Initialize PCA to reduce data to 2 components
pca = PCA(n_components=2)

# 3. Fit and transform the data
X_reduced = pca.fit_transform(X)

# 4. Display the reduced data and the explained variance
df_reduced = pd.DataFrame(data = X_reduced, columns =
['principal_component_1', 'principal_component_2'])
print("First 5 rows of data after PCA:")
print(df_reduced.head())
print(f"\nVariance explained by the 2 components:
{pca.explained_variance_ratio_.sum():.4f}")
```

Tab 3

As a data scientist, I sometimes want to explore different types of machine learning algorithms for different problems. For my reference, I created a list of the majority of ML algorithms. Hopefully this list can help others as well. The algorithms are grouped by category.

1. Supervised Learning

Supervised learning algorithms learn from labeled data, where the input-output pairs are known.

a. Regression Algorithms

- Linear Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression
- Support Vector Regression (SVR)

- Decision Tree Regression
- Random Forest Regression
- Gradient Boosting Regression (e.g., XGBoost, LightGBM, CatBoost)
- Bayesian Regression
- K-Nearest Neighbors (KNN) Regression

b. Classification Algorithms

- Logistic Regression
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN) Classification
- Decision Trees
- Random Forest Classification
- Gradient Boosting Machines (GBM)
- AdaBoost
- XGBoost
- LightGBM
- CatBoost

- Naive Bayes
- Neural Networks (e.g., Multilayer Perceptron)
- Quadratic Discriminant Analysis (QDA)
- Linear Discriminant Analysis (LDA)

2. Unsupervised Learning

Unsupervised learning algorithms learn from unlabeled data, identifying patterns and structures.

a. Clustering Algorithms

- K-Means
- Hierarchical Clustering
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
- Mean Shift
- Gaussian Mixture Models (GMM)
- BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)

- Affinity Propagation

b. Dimensionality Reduction Algorithms

- Principal Component Analysis (PCA)
- Independent Component Analysis (ICA)
- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Linear Discriminant Analysis (LDA)
- Factor Analysis
- Non-Negative Matrix Factorization (NMF)
- UMAP (Uniform Manifold Approximation and Projection)

3. Semi-Supervised Learning

Semi-supervised learning algorithms work with a small amount of labeled data and a large amount of unlabeled data.

- Self-training
- Co-training

- Generative Adversarial Networks (GANs) for Semi-Supervised Learning
- Graph-based Semi-Supervised Learning

4. Reinforcement Learning

Reinforcement learning algorithms learn by interacting with an environment to maximize cumulative rewards.

- Q-Learning
- SARSA (State-Action-Reward-State-Action)
- Deep Q-Networks (DQN)
- Policy Gradient Methods
- Actor-Critic Methods
- Deep Deterministic Policy Gradient (DDPG)
- Proximal Policy Optimization (PPO)
- Trust Region Policy Optimization (TRPO)

5. Ensemble Learning

Ensemble learning algorithms combine multiple models to improve performance.

- Bagging (Bootstrap Aggregating)
- Random Forest
- Boosting (e.g., AdaBoost, Gradient Boosting)
- Stacking
- Voting Classifier
- Blending

6. Neural Networks and Deep Learning

Deep learning algorithms involve multiple layers of neural networks.

- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory Networks (LSTM)
- Gated Recurrent Unit (GRU)
- Autoencoders

- Generative Adversarial Networks (GANs)
- Transformer Networks
- BERT (Bidirectional Encoder Representations from Transformers)
- GPT (Generative Pre-trained Transformer)

7. Probabilistic and Statistical Models

These algorithms are based on statistical methods and probabilistic reasoning.

- Bayesian Networks
- Hidden Markov Models (HMM)
- Markov Chain Monte Carlo (MCMC)
- Gaussian Processes

8. Instance-Based Learning

These algorithms memorize instances from the training data to make predictions.

- K-Nearest Neighbors (KNN)
- Locally Weighted Learning
- Case-Based Reasoning

9. Genetic Algorithms and Evolutionary Computing

These algorithms are inspired by the process of natural selection.

- Genetic Algorithms (GA)
- Genetic Programming (GP)
- Evolutionary Strategies (ES)
- Differential Evolution (DE)

10. Hybrid Methods

Combining multiple types of algorithms for better performance.

- Neural Evolution (combination of neural networks and genetic algorithms)

- Neuro-Fuzzy Systems (combination of neural networks and fuzzy logic)